

نظرة عامة على المشروع

لمعالجة PySpark في RDD (Resilient Distributed Dataset) هذا المشروع يهدف إلى تطبيق مفاهيم وتحليل بيانات الموظفين. تم تنفيذ 5 تمارين رئيسية تغطي عمليات التحويل والتصفية والتجميع على البيانات.

مصدر البيانات

البيانات موجودة في ملف نصي : /home/itversity/employees.txt

هيكل البيانات الأصلية (9 أعمدة):

- employee_id: معرف الموظف
- name: اسم الموظف
- Department: القسم
- job_title: المسمى الوظيفي
- salary: الراتب
- Location: الموقع
- hire_date: تاريخ التوظيف
- performance_rating: تقييم الأداء
- years_exp: سنوات الخبرة

مشكلة في البيانات: يوجد سطر تالف يحتوي على "Legal Counsel145000" بدلاً من "Legal Counsel, 145000"، وتم إصلاحه في دالة التنظيف.

إعداد بيئة العمل

```
python
```

```
from pyspark.sql import SparkSession
```

##إنشاء جلسة Spark

```
spark = SparkSession.builder \
```

```
.appName("RDD_Lab") \
```

```
.master("local[*]") \
```

```
.getOrCreate()
```

```
sc = spark.sparkContext
```

##تحديد مسار الملف

```
file_path = "file:///home/itversity/employees.txt"
```

🧹 تنظيف البيانات

تم كتابة دالة clean_line لتنظيف الأسطر وإصلاح الأخطاء:

```
python
```

```
def clean_line(line):
```

```
    """ تنظيف الأسطر وإصلاح الأخطاء """
```

```
    if not line or line.strip() == "":
```

```
        return None
```

إصلاح أي سطر به مشكلة

```
if "Legal Counsel145000" in line:
    line = line.replace("Legal Counsel145000", "Legal Counsel,145000")
parts = line.split(",")
if len(parts) >= 9:
    return line
return None

cleaned_rdd = text_rdd.map(clean_line).filter(lambda x: x is not None)
header = cleaned_rdd.first()
data_rdd = cleaned_rdd.filter(lambda x: x != header)

print(f"عدد السجلات بعد التنظيف: {data_rdd.count()}")
```

التمرين 1: تحويل البيانات إلى بنية منظمة

الهدف: تحويل كل سطر من النص إلى قائمة (Array) باستخدام `split` و `map`.

```
python

parsed_rdd = data_rdd.map(lambda x: x.split(","))
print("أول 5 سجلات بعد التحويل:")

for i, record in enumerate(parsed_rdd.take(5)):
    print(f" {i+1}. ID: {record[0]}, Name: {record[1]}, Dept: {record[2]}, Salary: ${record[4]}")
```

النتيجة المتوقعة:

text

1. ID: 1, Name: John Smith, Dept: Engineering, Salary: \$125000

2. ID: 2, Name: Sarah Johnson, Dept: Sales, Salary: \$85000

...

التمرين 2: حساب عدد مرات ظهور كل اسم

الهدف: استخدام map لاستخراج الأسماء و reduceByKey لتجميع التكرارات.

python

```
name_counts = data_rdd.map(lambda x: (x.split(";")[1], 1)) \
    .reduceByKey(lambda a, b: a + b)
```

```
print("عدد مرات ظهور كل اسم:")
```

```
for name, count in name_counts.collect():
```

```
    print(f" {name}: {count} مرة")
```

المنطق:

- map: يحول كل سطر إلى زوج (name, 1)
- reduceByKey: يجمع القيم لكل اسم

التمرين 3: تصفية السجلات غير الصالحة

الهدف: استخدام filter لإزالة السجلات ذات الراتب ≤ 0 .

python

```
valid_rdd = data_rdd.filter(lambda x: float(x.split(";")[4]) > 0)
```

```
print(f"السجلات الصالحة: {valid_rdd.count()}")
```

```
print(f"السجلات غير الصالحة: {data_rdd.count() - valid_rdd.count()}")
```

```
print("\nأول 5 سجلات صالحة:")
```

```
for line in valid_rdd.take(5):
```

```
    parts = line.split(";")
```

```
    print(f" {parts[0]} - {parts[1]} - {parts[2]} - ${parts[4]}")
```

التمرين 4: حساب متوسط الراتب لكل قسم

الهدف: حساب متوسط الراتب لكل قسم باستخدام mapValues و reduceByKey.

python

#استخراج القسم والراتب

```
dept_salary = valid_rdd.map(lambda x: (x.split(",")[2], float(x.split(",")[4])))
```

#حساب المجموع والعدد لكل قسم

```
dept_sum_count = dept_salary.mapValues(lambda x: (x, 1)) \
    .reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1]))
```

#حساب المتوسط

```
dept_avg = dept_sum_count.mapValues(lambda x: x[0] / x[1]) \
    .sortBy(lambda x: x[1], ascending=False)
```

```
print("متوسط الراتب لكل قسم:")
```

```
for dept, avg in dept_avg.collect():
```

```
    print(f" {dept}: ${avg:,.2f}")
```

المنطق:

1. استخراج (department, salary)
2. تحويل إلى (department, (salary, 1))
3. تجميع المجموع والعدد لكل قسم
4. حساب المتوسط (المجموع / العدد)
5. الترتيب تنازلياً حسب المتوسط

التمرين 5: حساب عدد الموظفين لكل قسم

الهدف: حساب عدد الموظفين في كل قسم باستخدام `map` و `reduceByKey`.

python

```
dept_count = valid_rdd.map(lambda x: (x.split(",")[2], 1)) \
    .reduceByKey(lambda a, b: a + b) \
    .sortBy(lambda x: x[1], ascending=False)
```

```
print("عدد الموظفين لكل قسم:")
total = valid_rdd.count()
for dept, count in dept_count.collect():
    percentage = (count / total) * 100
    print(f" {dept}: {count} موظف ({percentage:.1f}%)")
```

ملخص النتائج النهائي

python

```
print(f"""
{data_rdd.count()}
{valid_rdd.count()}
{data_rdd.count() - valid_rdd.count()}

{dept_count.first()[0]} ({dept_count.first()[1]} موظف)
{dept_avg.first()[0]} (${dept_avg.first()[1]:.2f})
""")
```

إجمالي السجلات في الملف
السجلات الصالحة
السجلات غير الصالحة

أعلى قسم من حيث عدد الموظفين
أعلى قسم من حيث متوسط الراتب

ملخص العمليات المستخدمة

التمرين	العمليات	الوظيفة
1	map, split	تحويل النص إلى بنية بيانات
2	map, reduceByKey	حساب التكرارات
3	filter	تصفية البيانات غير الصالحة
4	mapValues, reduceByKey	حساب المتوسطات
5	map, reduceByKey	حساب العدد لكل فئة

النقاط التعليمية المستفادة

1. **RDD الأساسية**: فهم كيفية إنشاء RDD من ملف نصي.
2. **التحويلات (Transformations)**: استخدام map, filter, reduceByKey, mapValues.
3. **الإجراءات (Actions)**: استخدام collect, count, take, first.
4. **تنظيف البيانات**: التعامل مع البيانات التالفة أو غير المكتملة.
5. **تحليل البيانات**: حساب التكرارات، المتوسطات، والنسب المئوية.

الخلاصة

تم بنجاح تنفيذ جميع التمارين الخمسة باستخدام RDD في PySpark:

- تحويل البيانات النصية إلى بنية منظمة
- حساب عدد مرات ظهور كل اسم
- تصفية السجلات غير الصالحة
- حساب متوسط الراتب لكل قسم
- حساب عدد الموظفين لكل قسم

هذا المشروع يوضح قدرة Spark على معالجة البيانات موزعياً (Distributed Processing) باستخدام RDD.